Fundamentos de la programación

Práctica 1. Pong

Indicaciones generales:

- La línea 1 del programa y siguientes deben contener los nombres de los alumnos de la forma (una línea por alumno):

  `// Nombre Apellido1 Apellido2`

- **Lee atentamente** el enunciado y desarrolla el programa tal como se pide, con la representación y esquema propuestos.
- El programa, además de correcto, debe estar bien estructurado y comentado. Se valorarán la claridad, la concisión y la eficiencia.
- La entrega se realizará a través del campus virtual, subiendo únicamente el archivo `Program.cs`, con el programa completo.
- El **plazo de entrega** finaliza el 2 de noviembre. No obstante se recomienda entregar antes de esa fecha para poder recibir comentarios y sugerencias de mejora.

Vamos a implementar una versión simple del clásico *Pong*, con el siguiente aspecto en pantalla:



El área de juego está limitada arriba y abajo por las franjas verdes, la línea central representa la red, la 'O' amarilla es la pelota y las barras verticales de los lados (roja y azul) representan las palas de los jugadores. La línea de texto de abajo muestra la puntuación de cada jugador.

La mecánica es sencilla: el juego comienza con la pelota en el centro del área de juego y se mueve en diagonal hacia uno de los jugadores. Cada jugador puede mover su pala arriba o abajo para interceptar la pelota. Cuando la pelota choca con una pala, rebota y cambia de dirección. Si la pelota pasa por detrás de la pala de un jugador, el otro jugador gana un punto y la pelota vuelve a aparecer en el centro del área de juego, moviéndose hacia el jugador que ha perdido el punto.

Vamos a desarrollar el juego gradualmente, razonando una representación adecuada para el estado del juego e implementando gradualmente las distintas funcionalidades. En cada paso se recomienda probar el programa para comprobar que funciona correctamente antes de pasar al siguiente fase.

1. Representación del estado del juego

Observemos que hay parámetros que, una vez fijados, no cambian durante la ejecución: el alto y el ancho del área de juego, y la longitud de las palas. Hay otros valores que sí cambian durante el juego: la posición vertical de las palas (coordenada Y), la posición de la pelota (con coordenadas X e Y), la dirección de la misma (vector con componentes X e Y) y los contadores de puntos de cada jugador. Por tanto, podemos representar el estado del juego como sigue:

```
// constantes (fuera del método Main)
const int ANCHO = 71, ALTO = 20,  // ancho y alto del área de juego
          LONG_PALA = 4;          // longitud de las palas

// variables (dentro de Main)
static void Main() {
      int pala1Y, pala2Y,         // posición vertical del extremo superior de las pal
          pelotaX, pelotaY,       // posición de la pelota
          dirX, dirY,            // dirección de la pelota
          puntos1, puntos2;       // puntos de cada jugador
```

Hay valores que también quedan fijados en función de los anteriores (y no se

hacen explícitos en el estado del juego), como la posición horizontal de las palas (1 y `ANCHO-2`) y la posición de la red (`ANCHO/2`). Por otro lado, por simpilidad, la dirección de la pelota está restringida a las diagonales, es decir, los vectores $(1, 1)$, $(1, -1)$, $(-1, 1)$ y $(-1, -1)$.

En el archivo `Program.cs` se proporciona una plantilla con estas declaraciones y el esquema general del programa.

2. Inicialización del estado

La posición vertical de las palas se inicializará de modo que las palas queden centradas en la columna correspondiente. La dirección de la pelota puede inicializarse con el vector $(1, 1)$, es decir, hacia abajo y hacia la derecha (hacia el jugador 2). Los puntos de cada jugador se inicializan a cero.

3. Renderizado en pantalla

Para mostrar el estado del juego en pantalla serán útiles los siguientes métodos de la clase `Console`:

- `Console.Clear()`: limpia la pantalla
- `Console.SetCursorPosition(left,top)`: sitúa el cursor en la posición `(left,top)` de la pantalla. La esquina superior izquierda de la pantalla es la posición `(0,0)`.
- `Console.BackgroundColor = ConsoleColor.Green` y `Console.ForegroundColor = ConsoleColor.Red`: cambian respectivamente el color del fondo a verde o y el color del texto a rojo (análogo con otros colores). Estos colores se mantienen hasta que se cambie a otro color o se restaure el color por defecto con `Console.ResetColor()`.
- `Console.ResetColor()`: restaura el color del texto al color por defecto

Para renderizar el estado del juego, limpiaremos la pantalla y dibujaremos los siguientes elementos en este orden:

- Muros superior e inferior, ambos de longitud `ANCHO`. El de arriba se sitúa en la línea 0 y el de abajo en `ALTO-1`. Cada bloque se dibuja como un espacio vacío, de color verde.
- Red central, de longitud `ALTO-2`, que se dibuja en la columna `ANCHO/2`, con el caracter `'|'`.
- Pelota, en la posición `(pelotaX,pelotaY)` con el caracter `'O'` en amarillo.
- Palas de los jugadores, de longitud `LONG_PALA`, que se dibujan en las columnas 1 y `ANCHO-2`, con el caracter `'|'` de color rojo y azul, respectivamente. Nótese que el valor `pala1Y` representa la posición vertical del extremo superior la pala1 (análogo para pala2). El resto de la pala se dibuja hacia abajo a partir de dicha posición.
- Puntos de cada jugador, que se escriben en la línea `ALTO` (justo debajo del área de juego)

Para probar el renderizado se puede mostrar el estado inicial del juego. Probar a cambiar los valores de las constantes y variables que representan el estado para comprobar que todo se dibuja correctamente en las posiciones correspondientes. Utilizar valores pequeños que faciliten la comprobación visual y posicionar las palas y la pelota en posiciones extremas.

4. Bucle principal: lectura de input de usuario y movimiento de las palas

En nuestra primera versión del bucle principal de juego vamos a leer el input de los jugadores desde teclado, actualizar la posición de las palas y

renderizar el nuevo estado. La pala1 se mueve arriba y abajo con las teclas `W`,`S` y la pala2 con `I`,`K`. Las palas no pueden salirse del área de juego, por lo que antes de actualizar su posición hay que comprobar que sus extremos no se salgan de los muros. La estructura del bucle principal es la siguiente:

```
// INICIALIZACIÓN DEL ESTADO DEL JUEGO

bool salir = false;
while (!salir) { // bucle principal
    // Lectura de teclado no bloqueante y movimiento de las palas
    if (Console.KeyAvailable) { // ver si se ha pulsado alguna tecla
        ConsoleKeyInfo tecla = Console.ReadKey(true); // recoger pulsación de tecla
        // ConsoleKey.W, ConsoleKey.S: actualizar posición pala1
        // ConsoleKey.I, ConsoleKey.K: actualizar posición pala2
        // ConsoleKey.Q: salir de juego
        // ConsoleKey.P: pausar
    }

    // RENDERIZADO

    // Retardo temporal para ralentizar la ejecución y que el juego sea jugable
    System.Threading.Thread.Sleep(90); // 90 milisegundos
}
```

En cada frame (en cada vuelta del bucle) se lee el input de los jugadores, se actualiza la posición de las palas y después se renderiza el nuevo estado. Nótese que si leyésemos el input con `Console.ReadLine()` el juego pararía hasta que el jugador escribiese algo y pulsase *intro*, y el juego no sería jugable. Por ello, es necesario hacer **lectura no bloqueante de teclado**: el juego continúa se pulse tecla o no. Para ello utilizamos las instrucciones `Console.KeyAvailable` y `Console.ReadKey(true)`: si hay pulsación de tecla se procesa y continúa la ejecución, y si no hay pulsación de tecla se continúa la ejecución sin hacer nada.

Para pausar el juego (con `'P'`) se puede hacer una lectura bloqueante con `Console.ReadLine()` que justo provoca la parada del juego hasta que el jugador

pulse *intro*.

Cuando se pruebe este bucle se observará que el renderizado provoca un parpadeo (flickering) que puede resultar molesto y dificultar el juego. En la sección de mejoras se explicará cómo reducir este efecto.

5. Movimiento de la pelota

Para mover a pelota basta con sumar el vector de dirección de la pelota a su posición en cada frame, justo después de la lectura de input y movimiento de las palas. Por ahora no tenemos colisiones, así que la pelota se moverá en diagonal hasta salirse del área de juego. En la siguente sección se explicará cómo detectar las colisiones y actuar en consecuencia.

6. Colisiones

Las pelota puede colisionar con los muros superior e inferior, con las palas de los jugadores o salirse del área de juego por los laterales. En cada frame comprobaremos si se ha producido alguna de estas colisiones y actuaremos en consecuencia.

Para detectar las colisiones de la pelota con los muros comprobaremos la posición vertical de la pelota y en caso de colisión se invierte la componente vertical de la dirección. La colisión con las palas se produce cuando la pelota está justo en la columna contigua a la pala (no sobre la pala) y en la misma posición vertical que alguna de las posiciones ocupadas por la pala; si hay colisión se invierte la componente horizontal de la dirección de la pelota.

Por último, para detectar el final de un tanto comprobaremos si sobrepasado la posición de alguna de las palas. En ese caso se incrementa el contador de puntos del jugador correspondiente y se reinicia la posición de la pelota en el centro del área de juego, con dirección hacia el jugador que ha perdido el punto.

Con esto ya tenemos una versión básica del juego que funciona correctamente. En la siguiente sección se explican algunas mejoras que se pueden implementar para mejorar la experiencia de juego.

7. Mejoras del juego

El juego admitirá varias mejoras que se pueden implementar una vez que la versión básica funcione correctamente. Algunas de ellas se explican a continuación.

7.1. Reducción del parpadeo (flickering) del renderizado

El renderizado en consola provoca un parpadeo que puede resultar molesto y dificultar el juego. Para reducir es este efecto debemos **evitar limpiar la pantalla** en cada frame (`Console.Clear()`) y dibujar todo de nuevo. En su lugar, podemos refrescar únicamente aquellos elementos que pueden cambia en el frame: la pelota y las palas.

Los muros superior e inferior no cambian, por lo que se dibujaran una sola vez antes del bucle principal y no se vuelven a dibujar. Para ello se elimina el código que los dibuja en el bucle principal.

Después, en el bucle principal para refrescar los elementos cambiantes, la idea es que primero se borran dichos elementos escribiendo espacios vacíos en sus antiguas posiciones y a continuación se dibujan en sus nuevas posiciones. Para ello, en cada vuelta del bucle habría que guardar las posiciones anteriores de la pelota y las palas en variables auxiliares. Un **truco** para conseguir este efecto evitando guardar posiciones anteriores es hacer el renderizado al final del bucle principal del siguiente modo:

- Renderizar la pelota y las palas en sus posiciones actuales.
- Poner el retardo temporal (`System.Threading.Thread.Sleep(90)`)

- Borrar la pelota y las palas escribiendo espacios vacíos en sus posiciones actuales. En el caso de la pelota, si esta se encuentra en la posición de la red, se debe (re-)dibujar el caracter `'|'` en lugar de un espacio vacío.

De este modo, la pelota y las palas se dibujan en sus posiciones actuales, se mantienen en pantalla durante el tiempo de retardo y luego se borran. En la siguiente vuelta del bucle (sin apenas lapso temporal) se calculan las nuevas posiciones y se repite el proceso.

7.2. Input múltiple de los jugadores

La recogida de input que hemos implementado permite que **un** jugador mueva su pala **una** posición en cada frame. Además una vez que un jugador ha pulsado una tecla, el otro no puede mover su pala en ese frame (en cierto modo, un jugador puede *robar* el turno). Para poder procesar varios inputs en un mismo frame basta con reemplazar `if (Console.KeyAvailable)` por `while (Console.KeyAvailable)`. De este modo ambos jugadores pueden mover su pala en el mismo frame y pueden moverla más de una posición si pulsan suficientemente rápido.

7.3. Otras mejoras opcionales. Velocidad de la pelota

Tal como está implementado el juego, la pelota se mueve siempre a la misma velocidad determinada por el retardo temporal que hemos fijado `System.Threading.Thread.Sleep(90)`. Es posible declarar una variable que almacene el retardo temporal y modificar su valor a lo largo del juego para que se mueva más rápido o más lento. Por ejemplo, se puede hacer que la velocidad de la pelota aumente gradualmente a medida que avanza el juego, disminuyendo el valor del retardo temporal.